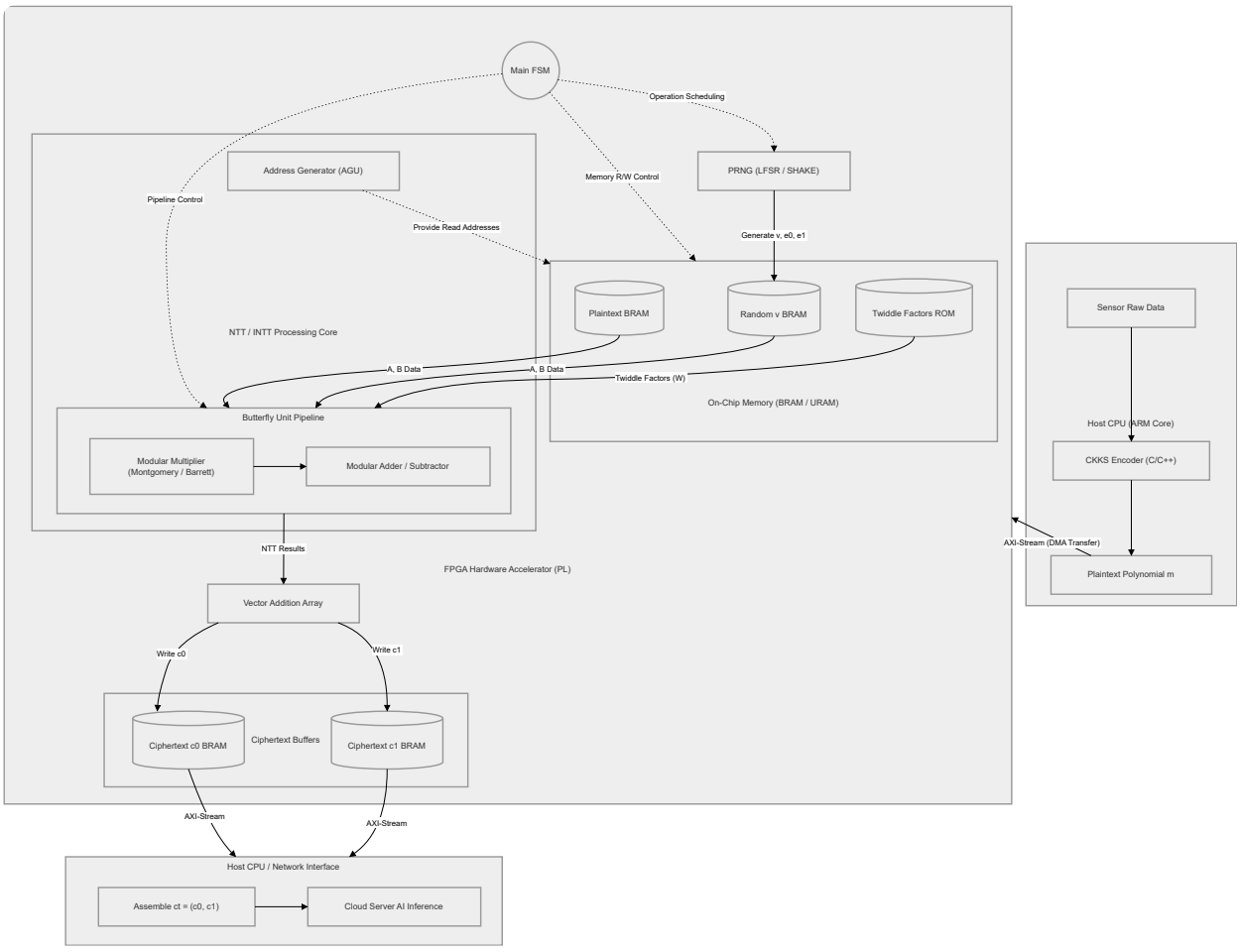
Project Proposal: EdgeFHE: A Lightweight HW/SW Co-designed Accelerator for Client-Side CKKS Operations on FPGAs

- **Auther: Ming Chen**
- **Architecture Diagram:**



- **Content Table:**

Contents

Project Proposal: EdgeFHE: A Lightweight HW/SW Co-designed Accelerator for Client-Side CKKS Operations on FPGAs

I. Topic Selection (Pain Points & Competitive Analysis)

1. What are the existing problems? (Pain Points)

2. What are others doing? (Feasibility Analysis & Literature Review)

3. How can we optimize or design? (Our Innovation)
II. Expected Final Deliverables
1. Minimum Viable Product (MVP)
2. Extensible Features & Integration
III. Implementation Challenges & Requirements
1. Resource Requirements List
2. Simplified System Architecture / Dataflow
3. Prerequisites & Skill Tree
IV. Appendix: Background knowledge
1. Non-Homomorphic Encryption vs. FHE (Conceptual Layer)
Non-Homomorphic Encryption (Traditional Encryption)
FHE, Fully Homomorphic Encryption 同態加密 (Our Focus)
Comparison Table
Reference
2. Foundation of FHE: LWE and RLWE (Math Foundation Layer)
LWE: Learning With Errors
RLWE: Ring Learning With Errors (Our Focus)
Comprehensive Table
Reference
3. The FHE Algorithm Family (Scheme Layer)
BGV / BFV: Exact Integer Arithmetic
CKKS: Approximate Floating-Point Arithmetic (Our Focus)
TFHE / FHEW: Fast Boolean Logic Gates
Comparison Table
Reference
4. The Performance Gatekeepers (Operational Layer)
Relinearization and Rescaling: Managing Ciphertext Growth**
Bootstrapping: The Ultimate Noise Reduction
Architectural Note for Our Edge-FPGA Design
5. The Hardware Acceleration Core (Implementation Layer)
Number-Theoretic Transform (NTT): Breaking the O(N2)O(N^2)O(N2) Compute Bound

Modular Reduction (Barrett / Montgomery): Bypassing Hardware Division

---

I. Topic Selection (Pain Points & Competitive Analysis)

1. What are the existing problems? (Pain Points)

The fundamental problem with conventional cloud-based AI computing is privacy. The standard workflow—Encrypted Transmission → Decryption at the Cloud → AI Inference → Encrypted Return—fully exposes sensitive data in plaintext during the decryption phase.

Fully Homomorphic Encryption (FHE) solves this by allowing computation directly on ciphertexts ($E(x) + E(y) = E(x + y)$). For AI applications, the **CKKS** scheme is the industry standard because it supports approximate arithmetic for floating-point numbers. However, executing FHE on edge devices (IoT Devices) encounters three fatal bottlenecks:

- **Extreme Computational Complexity (The Compute Bound):** The underlying mathematics relies on the Ring Learning With Errors (RLWE) problem. Encryption requires frequent execution of the Number-Theoretic Transform (NTT). On standard ARM processors, a single encryption takes several seconds due to massive **"Butterfly Operations"**.
- **Data Expansion and the Memory Wall:** A standard plaintext image (tens of KBs) explodes by a factor of 10x to 100x after CKKS encoding, becoming several MBs of ciphertext, creating immense memory bandwidth pressure.
- **Cloud-Edge Compute Asymmetry:** If the client (Edge device) cannot even perform the initial "Encryption" efficiently, the entire privacy-preserving AI ecosystem collapses.

---

2. What are others doing? (Feasibility Analysis & Literature Review)

Currently, leading tech corporations (Intel, IBM) and researchers are heavily focused on **Server-Side FHE acceleration** to solve complex noise-reduction (Bootstrapping) problems. For the limited research focusing on the client side, many attempts utilize **"Pure Hardware"** approaches—trying to build both floating-point Encoding and NTT Encryption entirely in Verilog. This results in excessive logic gate (LUT) consumption, making it impossible to deploy on cost-effective edge FPGAs.

References:

- *CKKS Foundation:* Cheon, J. H., et al. (2017). "Homomorphic encryption for arithmetic of approximate numbers." *ASIACRYPT*. Springer.
- *Client-Side Acceleration:* "Design and Analysis of Client-Side Operations in Homomorphic Encryption for FPGA Platforms." (2026).
- *Hardware Surveys:* Mera, J. M., et al. (2024). "A Survey on Hardware Accelerators for Fully Homomorphic Encryption." *IEEE Access*.

---

3. How can we optimize or design? (Our Innovation)

Instead of a pure hardware approach, our project proposes a **Lightweight Hardware/Software Co-designed Accelerator**.

- **Software (ARM Cortex):** Handles the floating-point IEEE 754 arithmetic for CKKS Encoding.
- **Hardware (FPGA Fabric):** Offloads the computationally heavy $O(N \log N)$ NTT operations via a custom-designed, highly pipelined cryptographic IP.

This strategic partitioning ensures our FPGA IP remains highly efficient, cost-effective, and perfectly suited for resource-constrained edge devices.

II. Expected Final Deliverables

1. Minimum Viable Product (MVP)

A functioning Hardware/Software co-designed CKKS Encryption System on an Edge FPGA System-on-Chip (SoC).

- **Hardware IP:** A pipelined NTT accelerator core synthesized via HLS (High-Level Synthesis) capable of performing polynomial modular arithmetic.
- **System Integration:** A bare-metal or Linux-based C/C++ driver running on the ARM core that successfully streams plaintext data to the FPGA via AXI-Stream and retrieves the encrypted ciphertext polynomials, demonstrating a significant speedup over pure software execution.

2. Extensible Features & Integration

- **End-to-End System Demo:** An interactive demonstration where a user inputs sensitive data (e.g., medical metrics) into a PC. The PC sends the data to the FPGA for rapid encryption. The ciphertext is then transmitted to a cloud server (or simulated locally) to run a simple Logistic Regression AI inference. The encrypted result is returned and decrypted, proving zero data leakage.
- **On-the-fly Twiddle Factor Generation:** To further reduce BRAM usage, implementing an algorithmic generator for NTT twiddle factors rather than storing them in massive ROM lookup tables.

III. Implementation Challenges & Requirements

1. Resource Requirements List

- **Hardware:** An FPGA SoC development board (e.g., Xilinx Zynq-7000 series like PYNQ-Z2, or Kria KV260 for larger capacity).
- **Software/Tools:** Xilinx Vivado Design Suite & Vitis HLS.
  - OpenFHE (C++) or TenSEAL (Python) for software validation and generating test vectors.
  - Linux environment for cross-compilation.

2. Simplified System Architecture / Dataflow

1. **Host CPU (ARM):** Receives raw sensor data $\rightarrow$ Performs CKKS Encoding $\rightarrow$ Generates plaintext polynomial $m(x)$.
2. **DMA Transfer:** Streams $m(x)$ via `AXI-Stream` to the FPGA Programmable Logic (PL).
3. **FPGA Crypto Engine:** * **PRNG:** Generates noise polynomials e_0, e_1.
   - **NTT Core:** Performs Butterfly operations with Modulo Arithmetic.
   - **Result:** Calculates ciphertexts $c_0(x)$ and $c_1(x)$.
4. **Output:** Streams ciphertexts back to the CPU via `AXI-Stream` for network transmission.

3. Prerequisites & Skill Tree

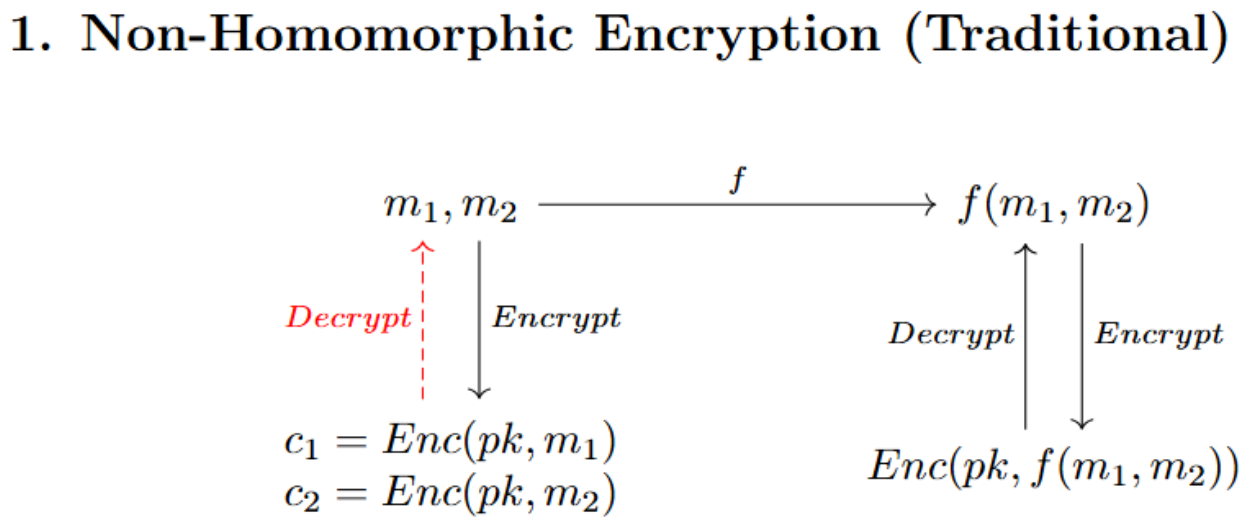
- **Theoretical Knowledge (Mathematics & Cryptography):**

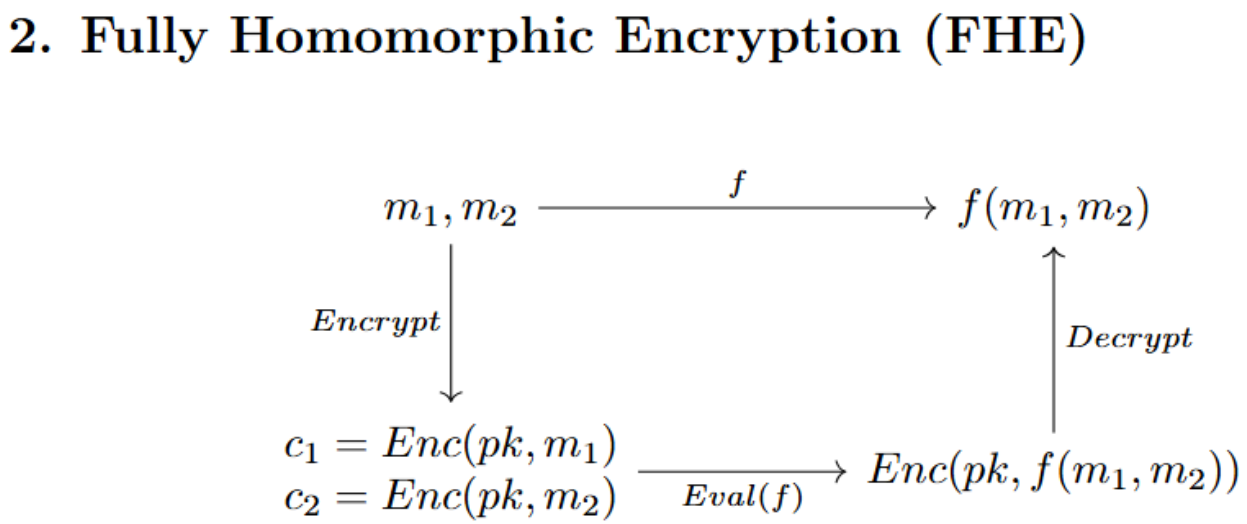
- *Polynomial Ring Arithmetic:* Understanding the foundational concept of $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$.
- *Number-Theoretic Transform (NTT):* Comprehending the Cooley-Tukey decimation framework.

- **Implementation Techniques (Hardware Design):**
  - *High-Level Synthesis (HLS):* Using C/C++ to describe hardware algorithms instead of raw Verilog to accelerate development.
  - *Hardware Architecture Optimization:* Mastering `#pragma HLS PIPELINE` and `#pragma HLS UNROLL` to manage loop unrolling and pipelining.
  - *AXI Interconnect Protocols:* Utilizing AXI-Stream and AXI-Lite to manage memory bandwidth and avoid data starvation.

IV. Appendix: Background knowledge

1. Non-Homomorphic Encryption vs. FHE (Conceptual Layer)





Non-Homomorphic Encryption (Traditional Encryption)

The fundamental limitation of computing on encrypted data.

Traditional encryption schemes, such as AES and RSA, are highly effective at securing data **at rest** (stored on a hard drive) and **in transit** (moving across a network). However,

they are fundamentally **"non-homomorphic"**. This means that mathematical or logical operations cannot be performed directly on the ciphertext.

If a client wants to leverage a cloud server to process encrypted data, the server must first possess the secret key to **decrypt the ciphertext back into plaintext**. After performing the computation, the server re-encrypts the result and sends it back. This mandatory decryption step creates a critical security vulnerability: the server, or any attacker who breaches the server, gains full access to the sensitive plaintext data during computation.

FHE, Fully Homomorphic Encryption 同態加密 (Our Focus)

The paradigm shift toward privacy-preserving computation.

Unlike traditional methods, FHE enables arbitrary arithmetic and logical computations directly on **encrypted data (ciphertext)** without ever requiring a decryption key.

In an FHE-enabled system, a client encrypts their sensitive data locally and sends the ciphertext to an untrusted cloud server. The server acts **"blindly"**, applying complex algorithms—such as Neural Network inference—directly to the encrypted payload. The server then returns an encrypted result. The data owner is the only party capable of decrypting this final result using their secret key. By eliminating the need for decryption during processing, FHE guarantees true **end-to-end data privacy**, making it the foundational technology for secure cloud computing and privacy-preserving AI.

Comparison Table

Feature	Traditional Encryption (Non-Homomorphic)	Fully Homomorphic Encryption (FHE)
Data State During Computation	Decrypted (Plaintext)	Encrypted (Ciphertext)
Server Trust Level Required	High Trust (Server holds the key)	Zero Trust (Server is blind to data)
Privacy Risk at Server	High (Vulnerable to data breaches)	None (End-to-End Privacy)
Computational Overhead	Very Low	High (Requires hardware acceleration)
Primary Use Case	Secure storage and secure data transfer	Privacy-preserving AI and Secure Cloud Computing

Reference

- **Overview of Fully Homomorphic Encryption**
- **Homomorphic Encryption for Beginners**

---

2. Foundation of FHE: LWE and RLWE (Math Foundation Layer)

The security of modern FHE schemes, such as CKKS, BGV, and BFV, relies on the computational hardness of lattice-based problems. Specifically, they are built upon the **Learning With Errors (LWE)** problem and its algebraic variant, **Ring-LWE (RLWE)**.

These problems are considered secure even against quantum computers (**Post-Quantum Cryptography**).

LWE: Learning With Errors

Concept:

Without errors, solving a system of linear equations modulo q is trivial using Gaussian

elimination. The LWE problem introduces a small, random **"noise"** into the equations. This minor perturbation transforms a mathematically simple problem into an **NP-hard problem**.

Definition:
Let q be a prime modulus.

- **Secret Vector (s):** A secret vector $s \in \mathbb{Z}_q^n$.
- **Public Matrix (A):** A randomly generated matrix $A \in \mathbb{Z}_q^{m \times n}$.
- **Error Vector (e):** A small error vector $e \in \mathbb{Z}_q^m$, where each element is sampled from a specific error distribution χ (typically a discrete Gaussian distribution centered at 0).

The LWE instance is defined as the pair (A, b), where:

$$b = (A \cdot s + e) \pmod q$$

The Hardness Problem:
Given the public matrix A and the resulting vector b, the **Search LWE Problem** asks to find the secret vector s. Because of the unknown error e, traditional linear algebra fails, and finding s requires exponential time.

Drawbacks of standard LWE:
While highly secure, standard LWE is inefficient for practical FHE. The public key size (the matrix A) grows quadratically $O(n^2)$, and encrypting a single bit of data requires matrix-vector multiplication, making data transmission and computation extremely slow.

RLWE: Ring Learning With Errors (Our Focus)

Concept:
To solve the inefficiency of standard LWE, researchers introduced RLWE. Instead of using vectors and matrices, RLWE operates over **Polynomial Rings**. This algebraic structure allows us to pack multiple data points into a single polynomial (similar to SIMD instructions in hardware), drastically reducing key sizes and accelerating computation.

Mathematical Definition:
RLWE operates in a polynomial quotient ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$, where N is a power of 2 (e.g., $N = 4096$ or 8192), and q is the ciphertext modulus. Every element in this ring is a polynomial of degree less than N.

- **Secret Polynomial ($s(x)$):** A secret polynomial $s(x) \in \mathcal{R}_q$ with small coefficients.

- **Public Polynomial ($a(x)$):** A uniformly random polynomial $a(x) \in \mathcal{R}_q$.
- **Error Polynomial ($e(x)$):** A small error polynomial $e(x) \in \mathcal{R}_q$, sampled from the error distribution.

The RLWE instance is defined as the pair (a, b), where:

$$b(x) = (a(x) \cdot s(x) + e(x)) \pmod{q}$$

Hardware Advantage (NTT):
In standard LWE, matrix multiplication requires $O(n^2)$ operations. In RLWE, polynomial multiplication $a(x) \cdot s(x)$ can be accelerated using the **Number-Theoretic Transform (NTT)**, reducing the time complexity to $O(N \log N)$. This mathematical shortcut is the primary reason why FHE can be practically accelerated on FPGAs.

Comprehensive Table

Feature	Standard LWE	Ring-LWE (RLWE)
Algebraic Structure	Vectors and Matrices ($\mathbb{Z}_q^n$)	Polynomial Rings ($\mathbb{Z}_q[X]/(X^N+1)$)
Key Size	$O(n^2)$ (Huge, MBs to GBs)	$O(N)$ (Compact, KBs to MBs)
Multiplication Complexity	$O(n^2)$ (Standard inner product)	$O(N \log N)$ (Using NTT)
Data Packing	Single scalar/bit per ciphertext	N slots per ciphertext (Batching)
Hardware Bottleneck	Memory Bandwidth (Fetching matrices)	DSPs and BRAM (NTT butterfly operations)
Primary FHE Schemes	FHEW, TFHE (Fast logic gates)	BGV, BFV, **CKKS** (Tensor/AI math)

Reference

- **The Learning with Errors Problem**
- **Cryptography 101: Ring Learning With Errors**

- **RLWE-based distributed key generation and threshold decryption**

---

3. The FHE Algorithm Family (Scheme Layer)

While all modern FHE schemes share similar mathematical foundations, they are optimized for entirely different data types and computational paradigms. For hardware and systems engineering, the priority is understanding their operational characteristics and best use cases, rather than their cryptographic proofs.

The FHE ecosystem is primarily divided into three major branches:

BGV / BFV: Exact Integer Arithmetic

- **Core Concept:** The BGV and BFV schemes are designed exclusively for **exact computations** over integers or finite fields.
- **Characteristics:** In these schemes, mathematical operations yield absolutely precise results. However, there is zero tolerance for noise overflow; if the cryptographic noise exceeds a certain threshold, the decryption process will fail completely, resulting in corrupted data.
- **Best Use Cases:** Applications demanding absolute precision, such as secure financial transactions, banking databases, Private Set Intersection (PSI), and genomic sequence matching.

CKKS: Approximate Floating-Point Arithmetic (Our Focus)

- **Core Concept:** The CKKS scheme represents a paradigm shift by supporting **approximate arithmetic** for real and complex numbers (floating-point values).
- **Characteristics:** CKKS treats cryptographic noise not as a fatal error, but as a natural part of computational approximation. It introduces a **"Rescaling"** mechanism to truncate the noise and manage precision after multiplications. The decrypted result is not exact but carries a negligible error, functionally mirroring the behavior of standard floating-point operations.
- **Best Use Cases:** AI, ML, and statistical analysis. Because neural networks (like CNNs and Transformers) inherently possess a high degree of error tolerance, CKKS has

become the undisputed industry standard for privacy-preserving AI inference.

TFHE / FHEW: Fast Boolean Logic Gates

- **Core Concept:** Unlike BGV and CKKS, which pack arrays of numbers into polynomials (SIMD operations), TFHE (Fast Fully Homomorphic Encryption over the Torus) and FHEW operate at the bit level to evaluate individual **Boolean logic gates** (e.g., AND, OR, XOR, MUX).
- **Characteristics:** These schemes are renowned for their ultra-fast "Bootstrapping" (noise reduction) capabilities, allowing for the execution of infinitely deep logic circuits.
- **Limitations for AI:** While they excel at evaluating arbitrary control logic and state machines, they are highly inefficient at performing the massive, dense Multiply-Accumulate (MAC) operations required for matrix multiplications in modern AI workloads.

Comparison Table

Scheme Family	Supported Data Type	Computation Paradigm	Primary Application Domain	Suitability for Deep Learning
BGV / BFV	Integers / Finite Fields	Exact Arithmetic	Finance, Secure Databases	Very Low (Data expansion issues)
CKKS	Floating-point (Real/Complex)	Approximate Arithmetic	Machine Learning, AI Inference	**Extremely High (Standard)**
TFHE / FHEW	Bits / Booleans	Logic Gate Operations	Generic Logic Circuits, Smart Contracts	Low (Inefficient for MAC operations)

Reference

- **Homomorphic Encryption for Arithmetic of Approximate Numbers**
- **Fully Homomorphic Encryption without Bootstrapping**
- **Somewhat Practical Fully Homomorphic Encryption**
- **FHEW: Bootstrapping homomorphic encryption in less than a second**
- **TFHE: Fast Fully Homomorphic Encryption over the Torus**
- **What is CKKS ?**
- **A Survey on Hardware Accelerators for Fully Homomorphic Encryption**

---

4. The Performance Gatekeepers (Operational Layer)

While the underlying mathematical frameworks (like RLWE) guarantee security, specific cryptographic operations determine the practical feasibility and computational overhead of an FHE system. For hardware architects, understanding these operations is crucial, as they represent the primary bottlenecks in dataflow and memory management.

Relinearization and Rescaling: Managing Ciphertext Growth**

Homomorphic multiplication is notoriously expensive. When two ciphertexts are multiplied, it causes two critical issues that must be immediately addressed:

- **Relinearization (Controlling Dimension):** Multiplying two standard ciphertexts (each consisting of two polynomials) results in an expanded ciphertext containing three polynomials. If left unchecked, subsequent multiplications would cause the ciphertext size to grow exponentially, rapidly exhausting hardware memory. **Relinearization** is the operation that compresses this expanded ciphertext back down to its standard size, requiring complex key-switching matrices.
- **Rescaling (Controlling Precision & Noise):** Specific to the CKKS scheme, multiplying ciphertexts also squares the underlying scaling factor (e.g., $\Delta \times \Delta = \Delta^2$). **Rescaling** dynamically divides the ciphertext by a base modulus, effectively truncating the least significant bits. This brilliant mechanism acts exactly like rounding in standard floating-

point arithmetic—it discards the accumulated noise and prevents the numerical values from overflowing the hardware's bit-width limits.

Bootstrapping: The Ultimate Noise Reduction

Every homomorphic operation—particularly multiplication—injects a small amount of cryptographic noise into the ciphertext. If this noise surpasses a critical mathematical threshold, the message is permanently destroyed and cannot be decrypted.

- **The Mechanism: Bootstrapping** is the definitive solution to this problem. It is an incredibly complex operation where the server homomorphically evaluates the FHE decryption circuit *on the ciphertext itself*. This essentially "refreshes" the ciphertext, resetting the noise to a minimal baseline and allowing for computations of infinite depth.

Architectural Note for Our Edge-FPGA Design

- **Strategic Offloading:** Bootstrapping is exceptionally resource-intensive, often taking seconds to minutes per operation, and requires massive amounts of on-chip memory (BRAM/URAM). **Therefore, our proposed edge-FPGA accelerator intentionally excludes bootstrapping hardware.**
- **System Partitioning:** In our privacy-preserving IoT architecture, the edge FPGA is strictly responsible for ultra-fast, lightweight Encoding and Initial Encryption (NTT acceleration). The heavy computational burden of Homomorphic Evaluation and Bootstrapping is strategically offloaded to the cloud server. This HW/SW partitioning ensures our FPGA IP remains highly efficient, cost-effective, and perfectly suited for resource-constrained edge devices

---

5. The Hardware Acceleration Core (Implementation Layer)

This layer represents the primary engineering contribution of our project. It focuses on translating abstract cryptographic mathematics into highly optimized physical hardware architectures (via RTL or High-Level Synthesis). To achieve real-time client-side encryption on an edge FPGA, our custom IP core specifically targets the two most computationally expensive bottlenecks in the CKKS pipeline:

Number-Theoretic Transform (NTT): Breaking the $O(N^2)$ Compute Bound

- **The Problem:** Standard polynomial multiplication (convolution) is the most time-consuming operation in RLWE-based cryptography. For a polynomial of degree N, a naive multiplication requires $O(N^2)$ operations. Given that FHE typically requires $N \geq 4096$, this approach is fundamentally unscalable and leads to massive latency.
- **The Hardware Solution:** The **Number-Theoretic Transform (NTT)** is the cryptographic equivalent of the Fast Fourier Transform (FFT) operating over finite fields. By transforming the polynomials into the NTT domain, the complex convolution is simplified into element-wise (point-wise) multiplications, drastically reducing the time complexity to $O(N \log N)$.
- **Architectural Implementation:** In our FPGA design, this translates to constructing highly pipelined **Cooley-Tukey Butterfly Units (BUs)** combined with sophisticated Address Generation Units (AGUs) to manage the complex bit-reversal and strided memory access patterns without stalling the pipeline.

Modular Reduction (Barrett / Montgomery): Bypassing Hardware Division

- **The Problem:** In the $\mathbb{Z}_q$ polynomial ring, every single arithmetic operation (addition or multiplication) must be followed by a modulo operation ($(\mathrm{mod}\ q)$). In digital logic design, standard division (using the `%` operator) is notoriously inefficient, consuming excessive routing resources, high-latency LUTs, and DSP slices.
- **The Hardware Solution:** To overcome this critical bottleneck, our hardware accelerator implements fast modular reduction algorithms, specifically **Barrett Reduction** or **Montgomery Reduction**.
- **Architectural Implementation:** These elegant algorithms replace the costly hardware division entirely with a combination of pre-computed constants, low-cost bit-shifts ($\gg$), and simple multiplications. This optimization is the absolute key to achieving a high clock frequency and allowing our data path to process one coefficient per clock cycle (Initiation Interval, II = 1).